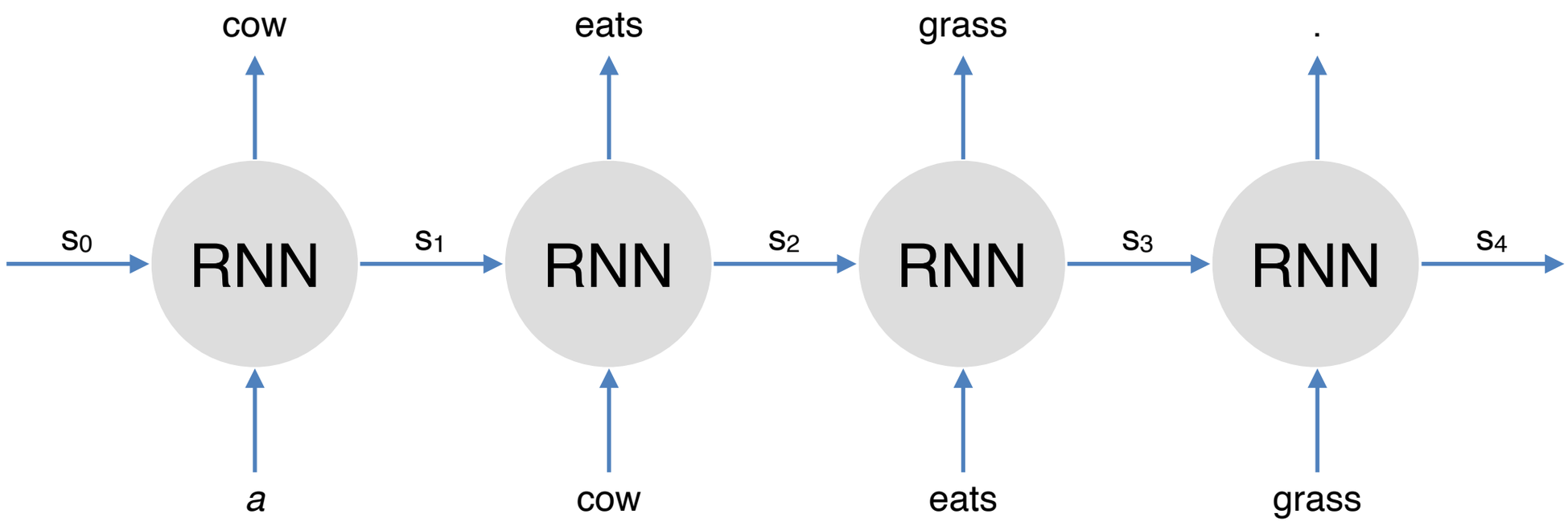
# COMP90042 NLP

# L9: Transformer

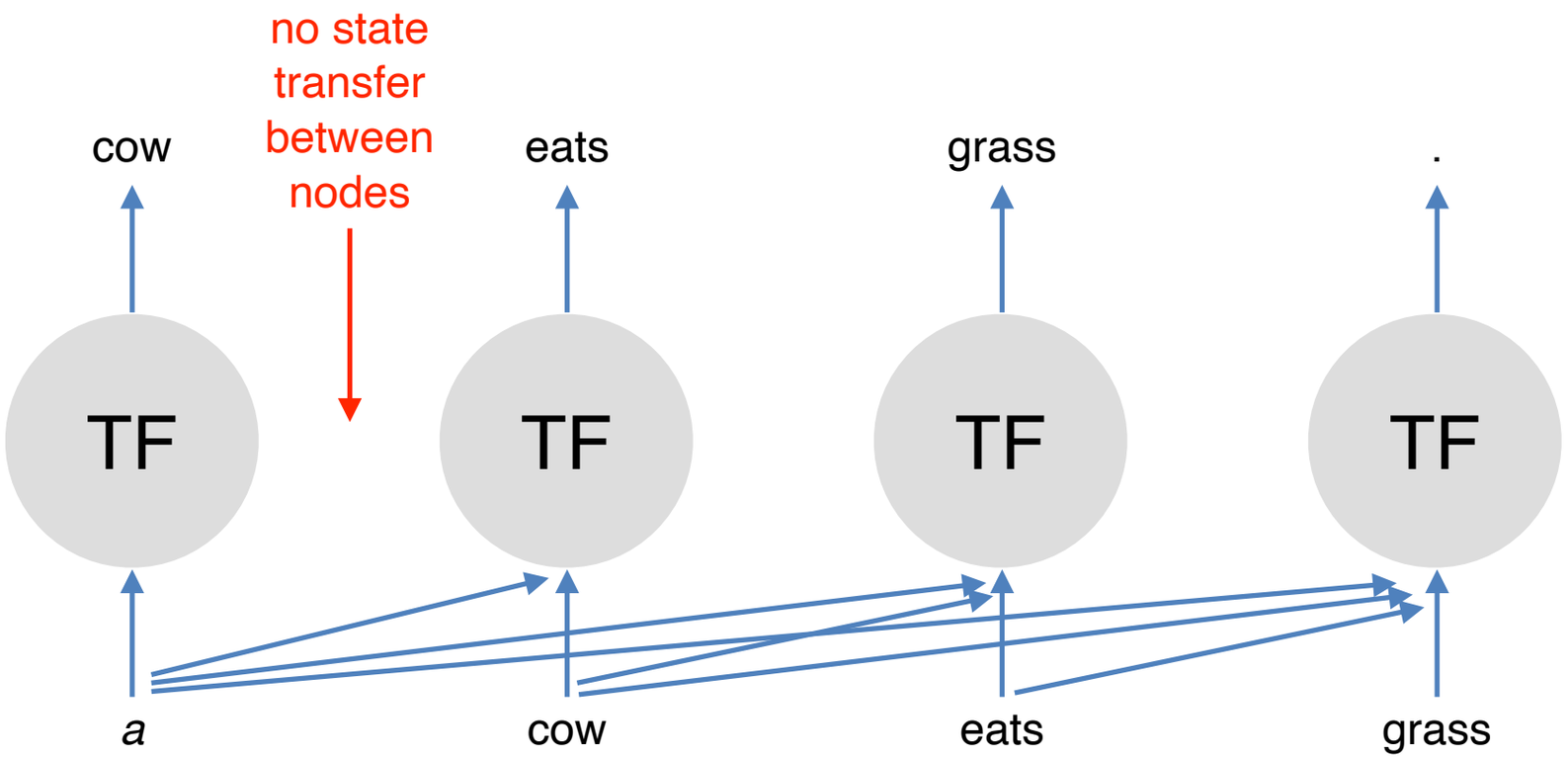

Jey Han Lau

# RNN Language Model

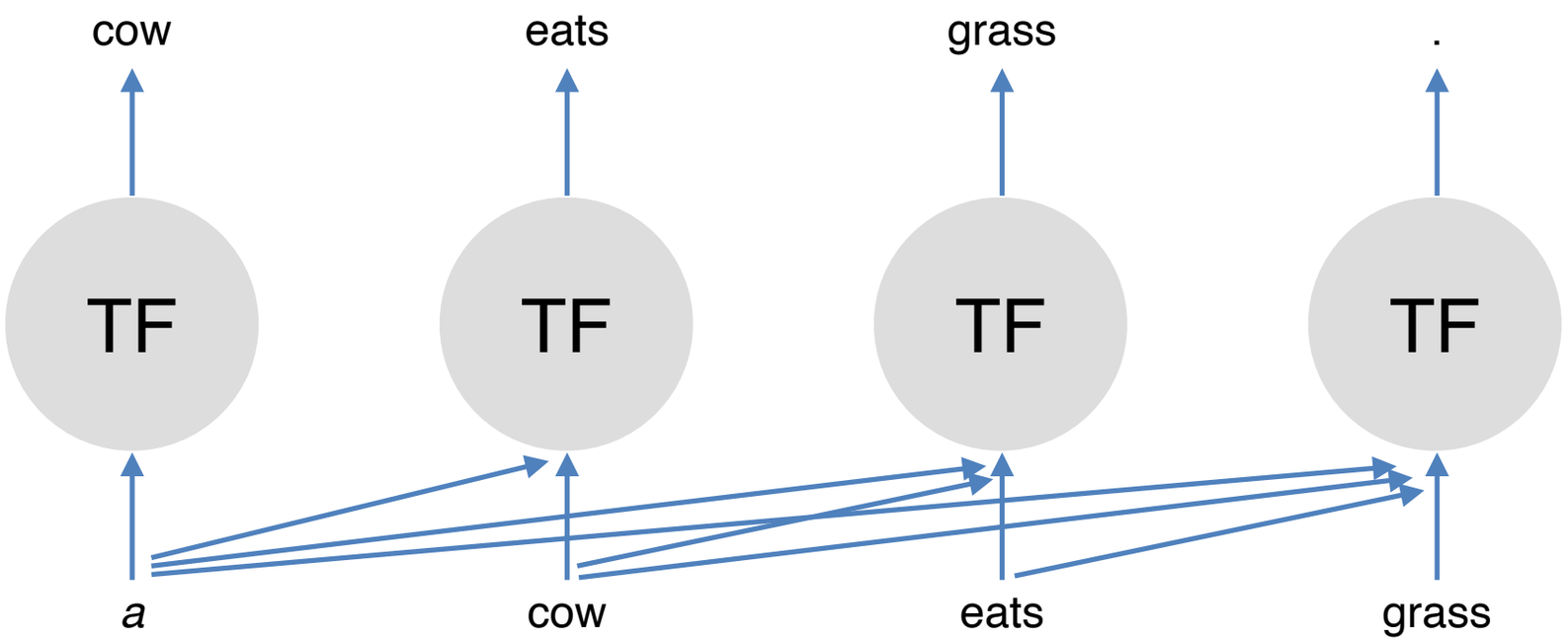



- Recurrent function: we have to proceed sequentially, one word at a time
- To train it to predict "grass", it must first complete its prediction for the previous words ("cow" and "eats")

# Transformer



# Transformer




- Each prediction is independent of each other
- Computation can be **parallelised**
- How? Use attention

# Outline

- Attention with Query/Key/Value Comparison
- Transformer Block
- Positional Embeddings
- Training

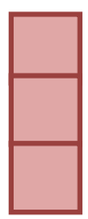
# Attention with Query/Key/Value Comparison

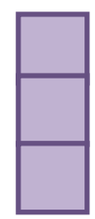
# Attention is All You Need

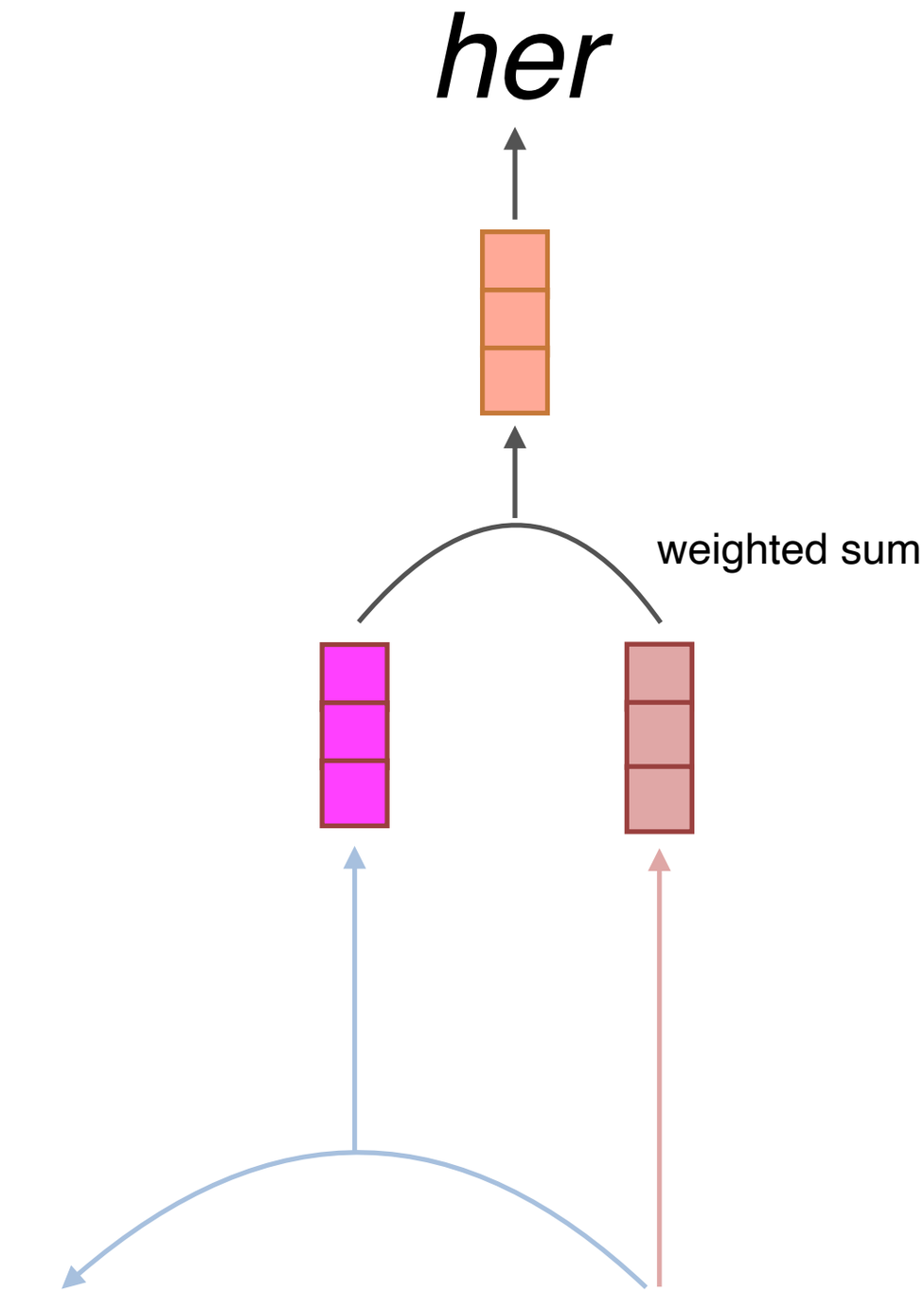
- Vaswani et al. (2017): https://arxiv.org/abs/1706.03762
- Use **attention** to capture dependencies between words

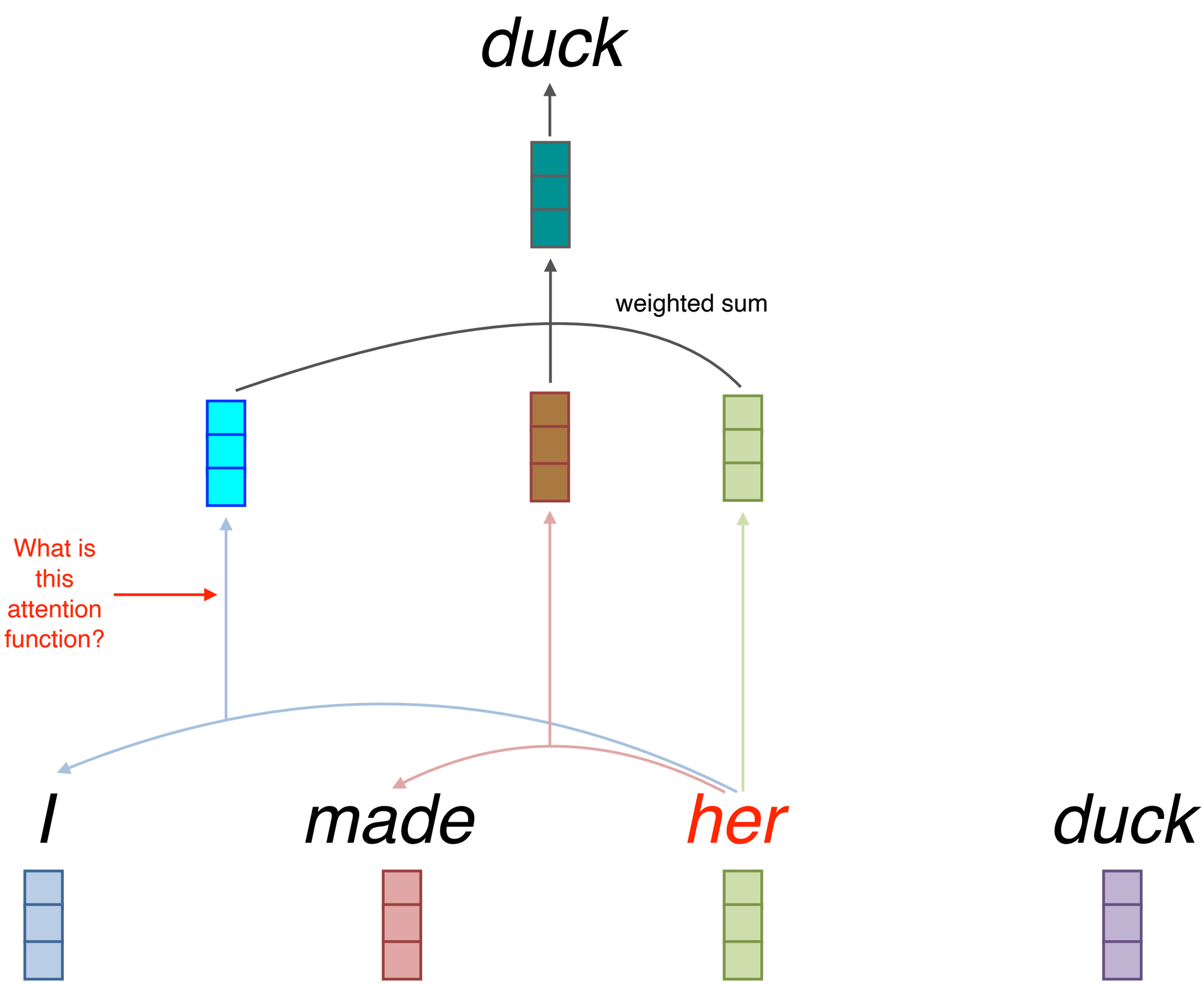
*made*

*I* *made* *her* *duck*






her
weighted sum
I
made
her
duck

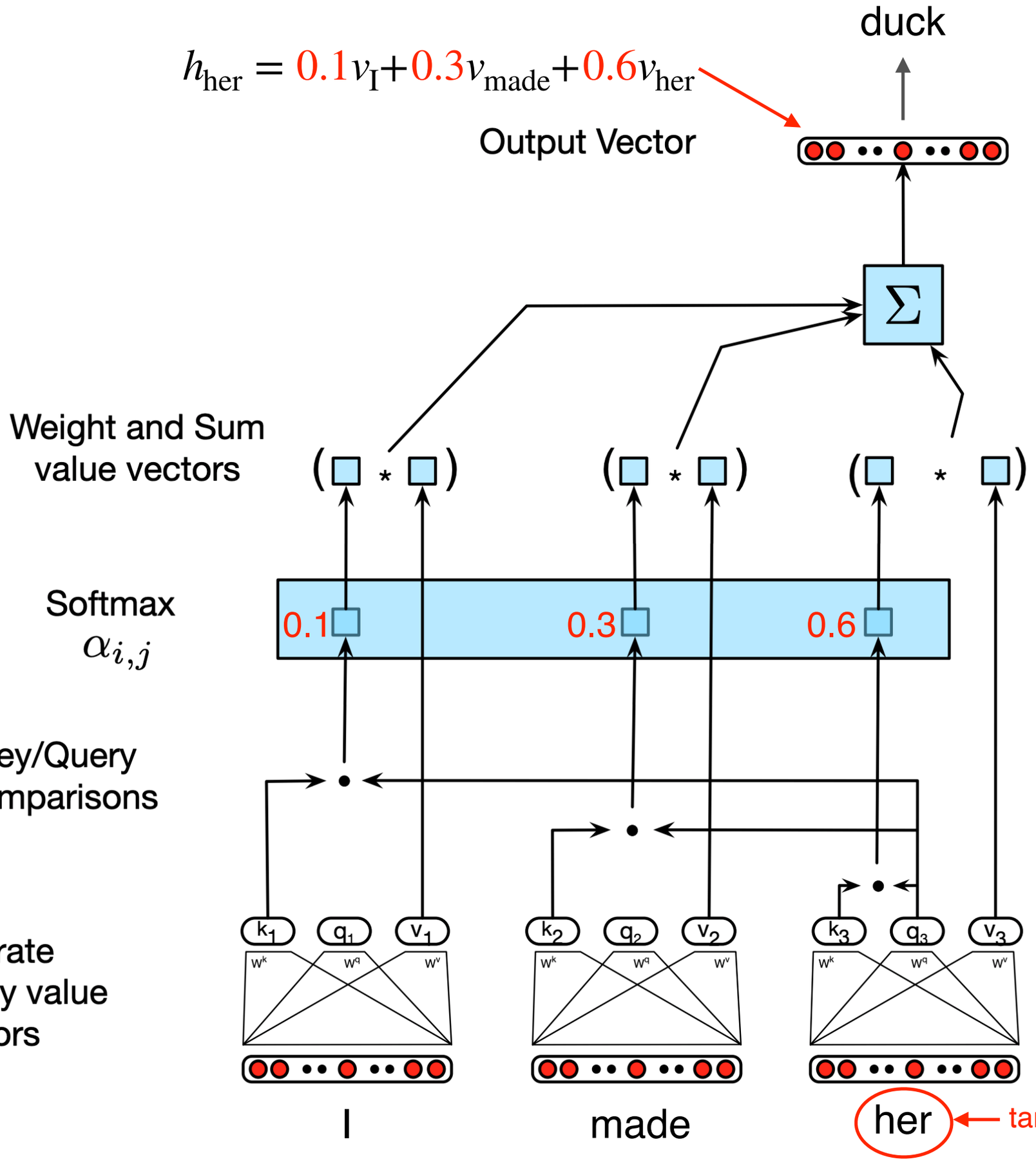

duck
weighted sum
What is this attention function?
I
made
her
duck

# Self-Attention with Query, Key, Value Comparison

- Input:
  - query q (e.g. *her* )
  - key k and value v (e.g. *I*, *made*)
- Query, key and value are all **vectors**
  - linear projections from embeddings
- Comparison between **query vector of target word** (*her*) and **key vectors of context words** (*made*) to compute **weights**
- Final representation of target word = weighted sum of **value vectors of context words and target word**

$$h_{\text{her}} = 0.1v_{\text{I}} + 0.3v_{\text{made}} + 0.6v_{\text{her}}$$

$$h_{\text{her}} = 0.1v_{\text{I}} + 0.3v_{\text{made}} + 0.6v_{\text{her}}$$



# Self-Attention

$$\text{SelfAttention}(q, K, V) = \sum_i \frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}} \times v_i$$

$e^{q \cdot k_i}$ ← query and key comparisons

$\frac{e^{q \cdot k_i}}{\sum_j e^{q \cdot k_j}}$ ← softmax

$h_{\text{her}} \in \mathbb{R}^{1 \times d_v}$

- Multiple queries, stack them in a matrix

$$\text{SelfAttention}(Q, K, V) = \text{softmax}(QK^T)V$$

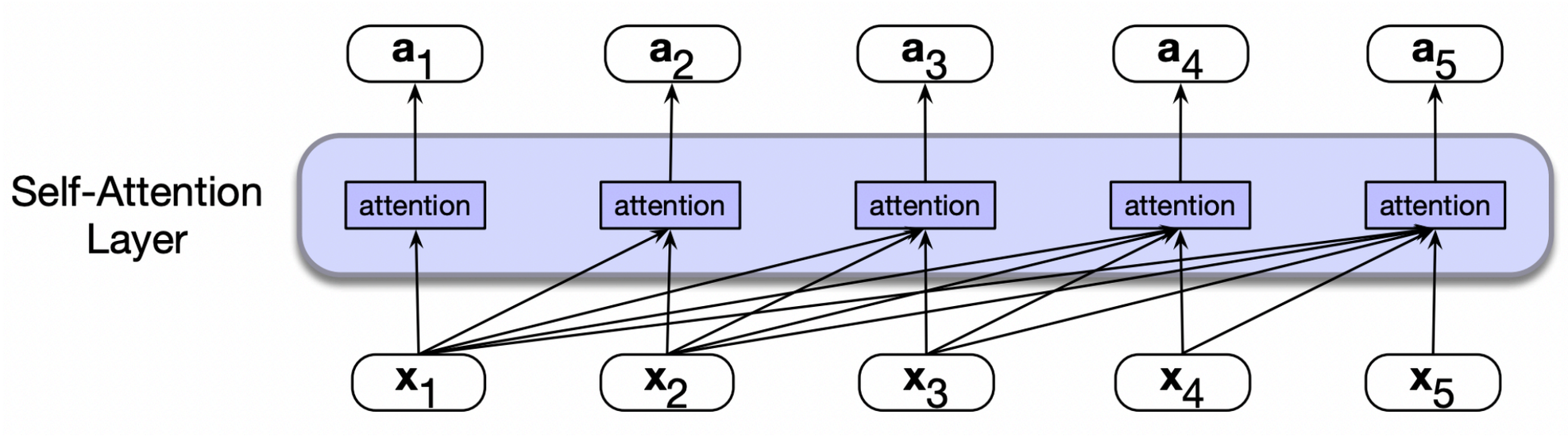
- Uses **scaled dot-product** to prevent values from growing too large

$$\text{SelfAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \in \mathbb{R}^{N \times d_v}$$

$\sqrt{d_k}$ ← dimension of query and key vectors

# Contextual Representation

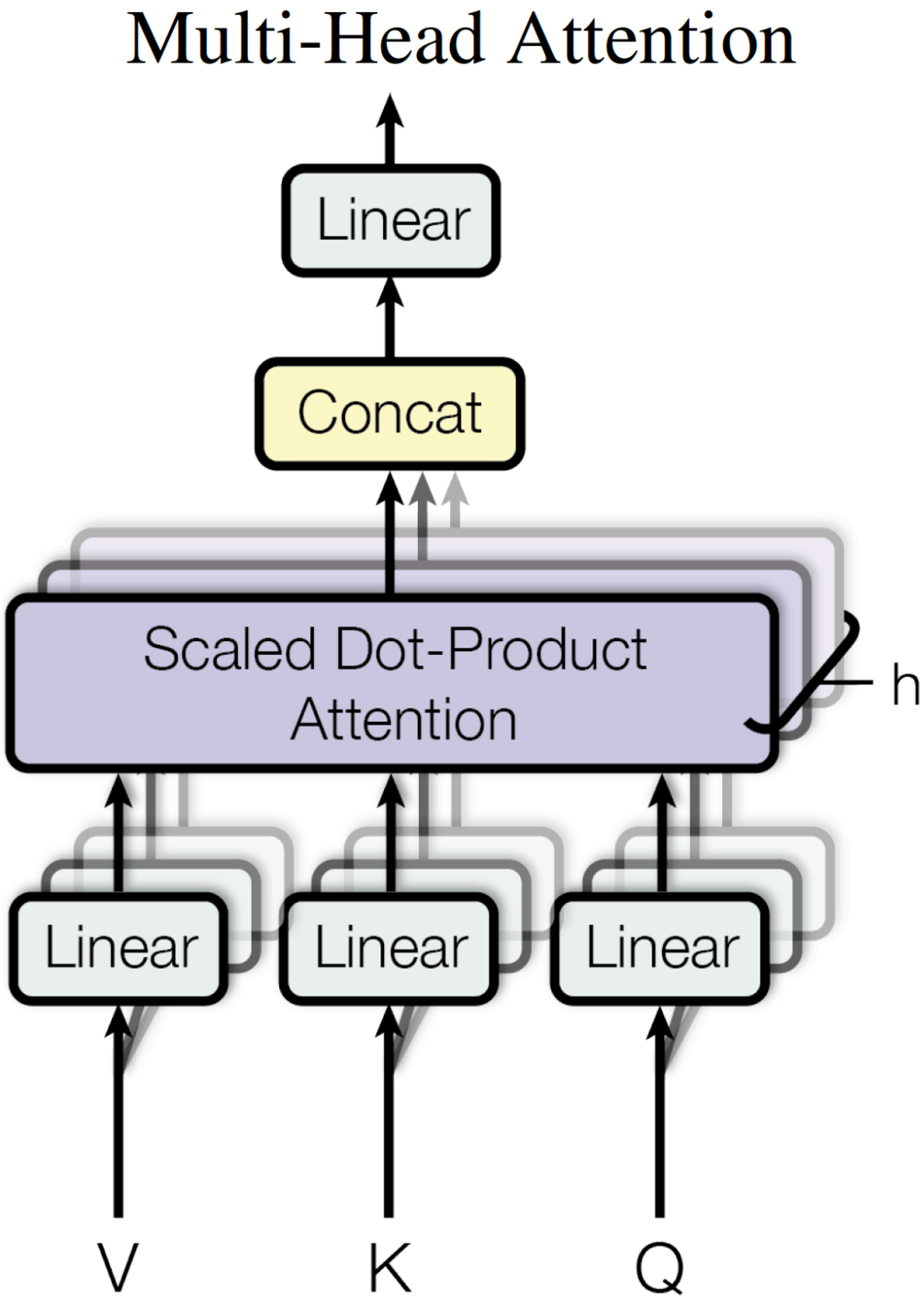
- Self-attention builds **contextual representation** for each word in the sentence




- That is, a representation that takes its neighbouring context into consideration

# Multihead Attention

- Different words in a sentence can relate to one another in many ways
  - syntactically (noun-verb agreement)
  - semantically (synonyms)
  - discourse (*it* refers to the dog earlier)
- A single attention can't capture all these relationships
- Multihead Attention!




Do the attention computation multiple times, all independently!

now, each embedding captures information about itself AND preceding words

[N x d]

$a_1$ $a_2$ $a_3$ … $a_N$

Project from $hd_v$ to d

$\mathbf{w}^O$ $[hd_v \times d]$

Concatenate Outputs $[N \times hd_v]$

Each word's output embedding has a dimension of $hd_v$

Multihead Attention Layer with h=4 heads

head1 output val $[N \times d_v]$ | head2 output val $[N \times d_v]$ | head3 output val $[N \times d_v]$ | head4 output val $[N \times d_v]$

$\mathbf{w}^Q_4, \mathbf{w}^K_4, \mathbf{w}^V_4$ Head 4

$\mathbf{w}^Q_3, \mathbf{w}^K_3, \mathbf{w}^V_3$ Head 3

$\mathbf{w}^Q_2, \mathbf{w}^K_2, \mathbf{w}^V_2$ Head 2

$\mathbf{w}^Q_1, \mathbf{w}^K_1, \mathbf{w}^V_1$ Head 1

[N x d]

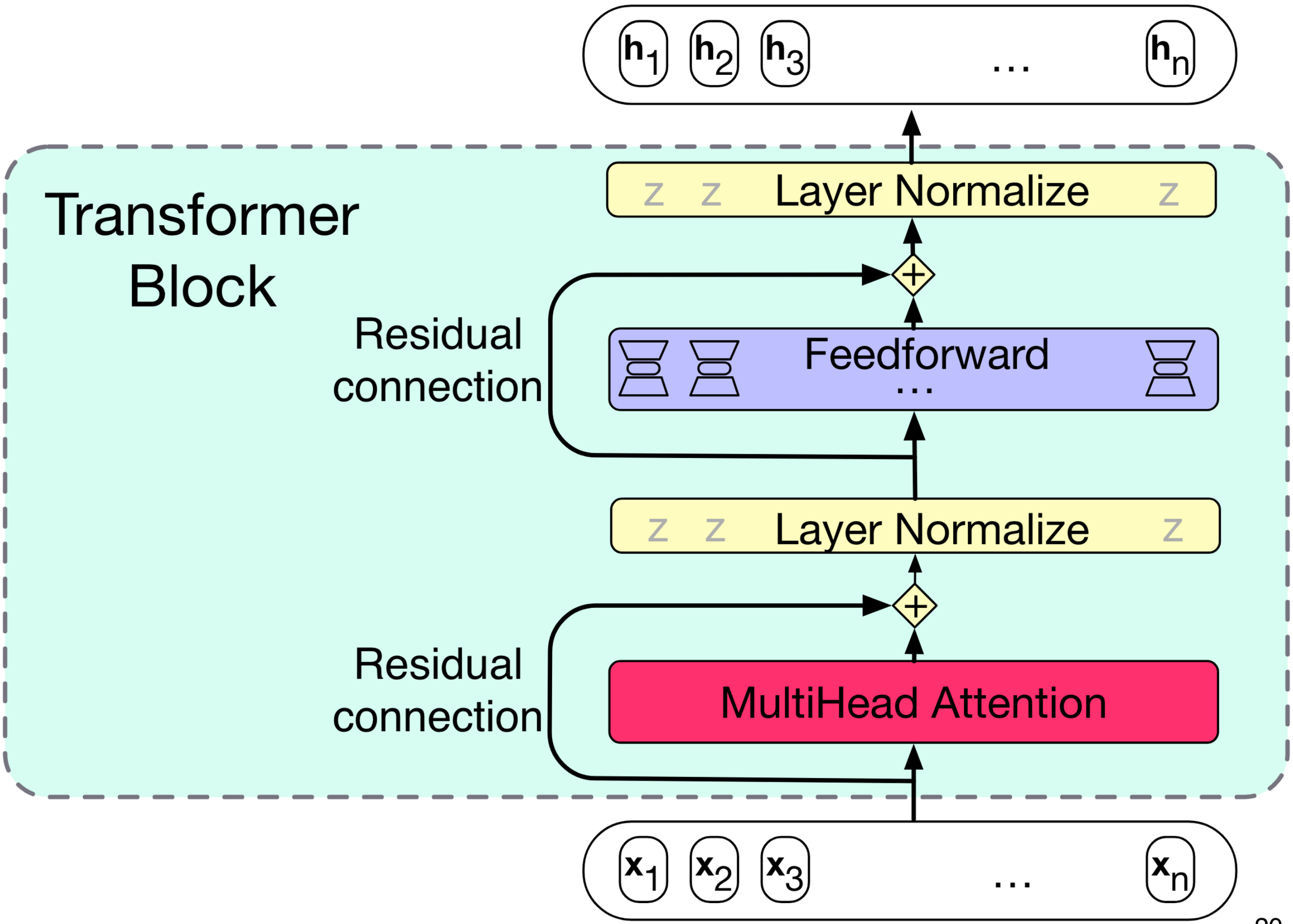
$x_1$ $x_2$ $x_3$ … $x_N$

each word embedding captures only information about itself

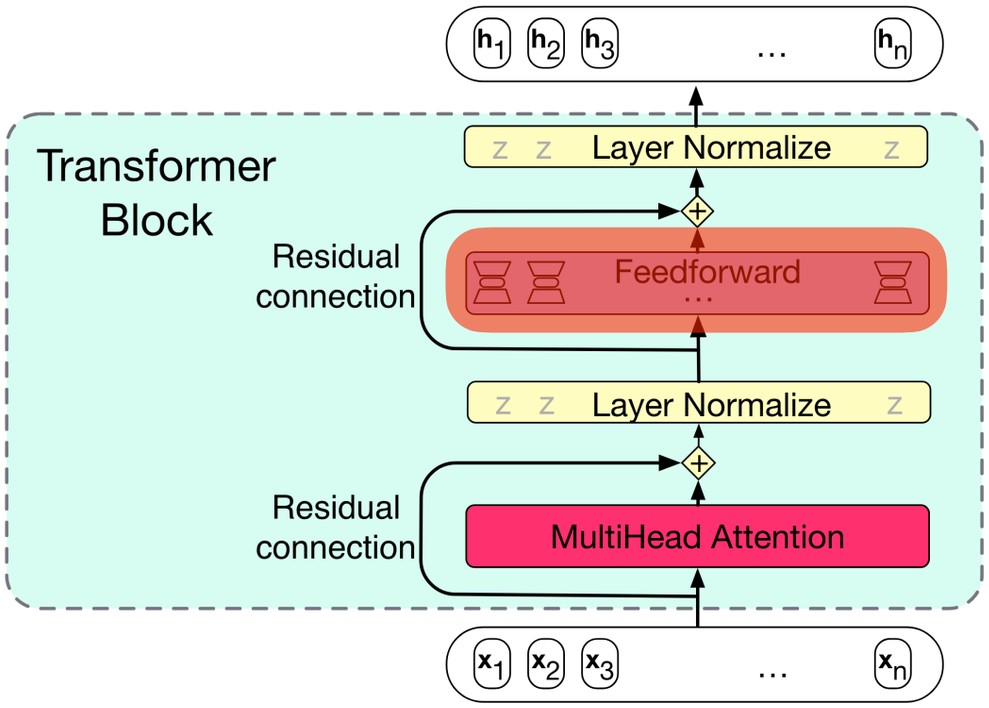
# Transformer Block

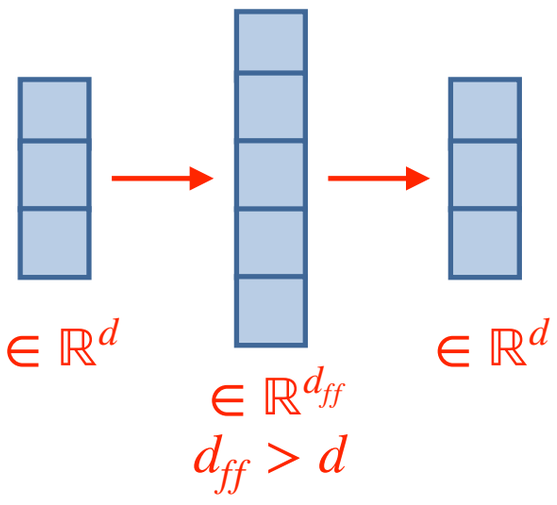
# Transformer Block

- But, there's more!
- In addition to self-attention, a full "transformer block" also has:
  - 1x feedforward layer
  - 2x residual connections
  - 2x normalising layers


$h_1$ $h_2$ $h_3$ … $h_n$
Transformer Block
z z Layer Normalize z
Residual connection
Feedforward
…
z z Layer Normalize z
Residual connection
MultiHead Attention
$x_1$ $x_2$ $x_3$ … $x_n$

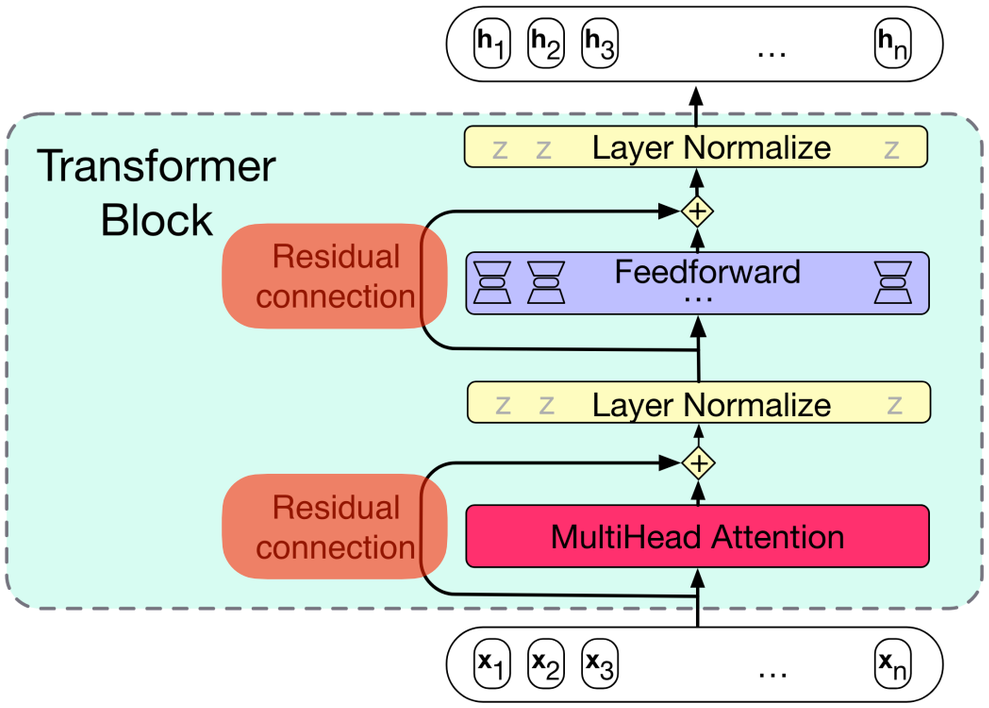
# Feedforward




- 2 layer network
- An embedding is expanded then compressed again
- Same projection matrix for each word position

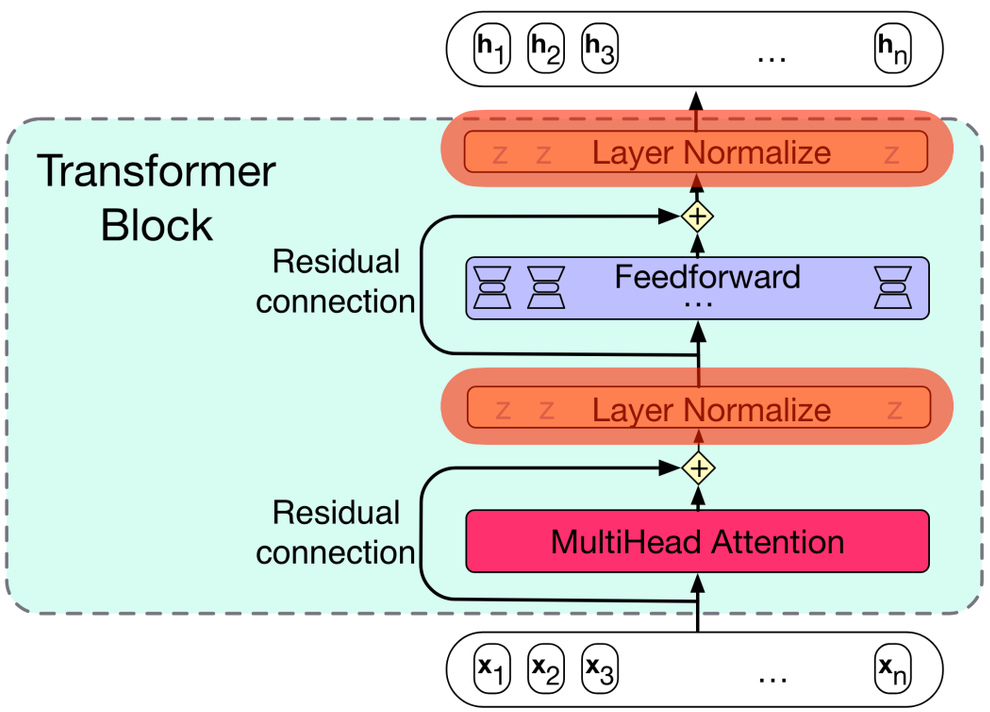
# Residual Connections




- Connections that pass information from lower layer to higher layer, **without going through the intermediate layer**
- Gives higher level layer direct access to lower layer → improves learning

- In transformer, input is simply added to the output
- 1st residual connection: word embedding added to the output embedding of multihead attention

# Normalising Layer ("Layer Norm")




- A form of regularisation
- Layer norm is a variation of z-score, **applied to a single vector**

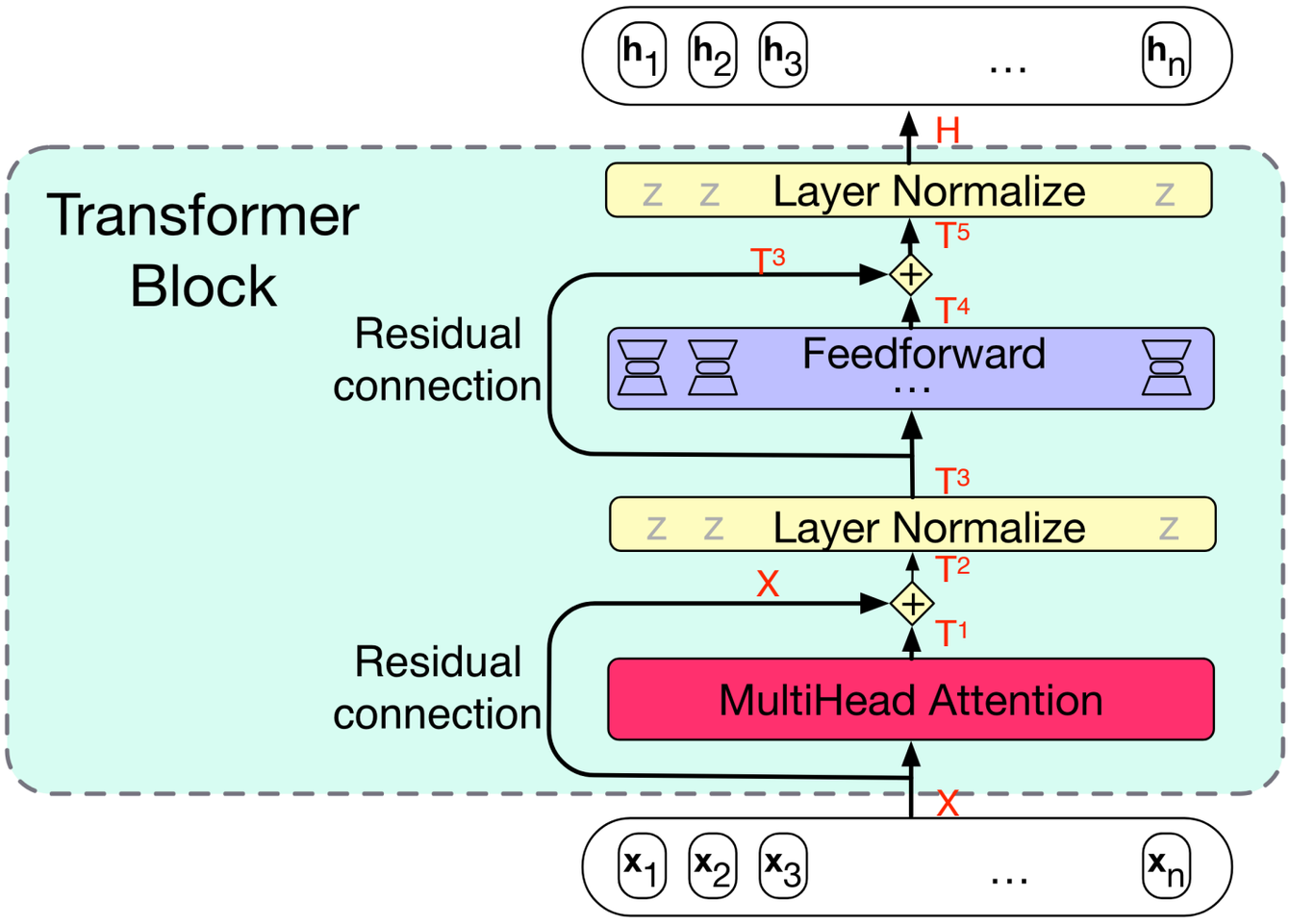
1. Computes *mean* and *std* given the elements of a vector ($x$)
2. Normalises each element based on *mean* ($\mu$) and *std* ($\sigma$)

$$\hat{x} = \frac{x - \mu}{\sigma}$$

$$LayerNorm = \alpha\hat{x} + \beta$$

two learnable parameters

# Putting Everything Together




$$T^1 = \text{MultiHeadAttention}(X)$$

$$T^2 = X + T^1$$

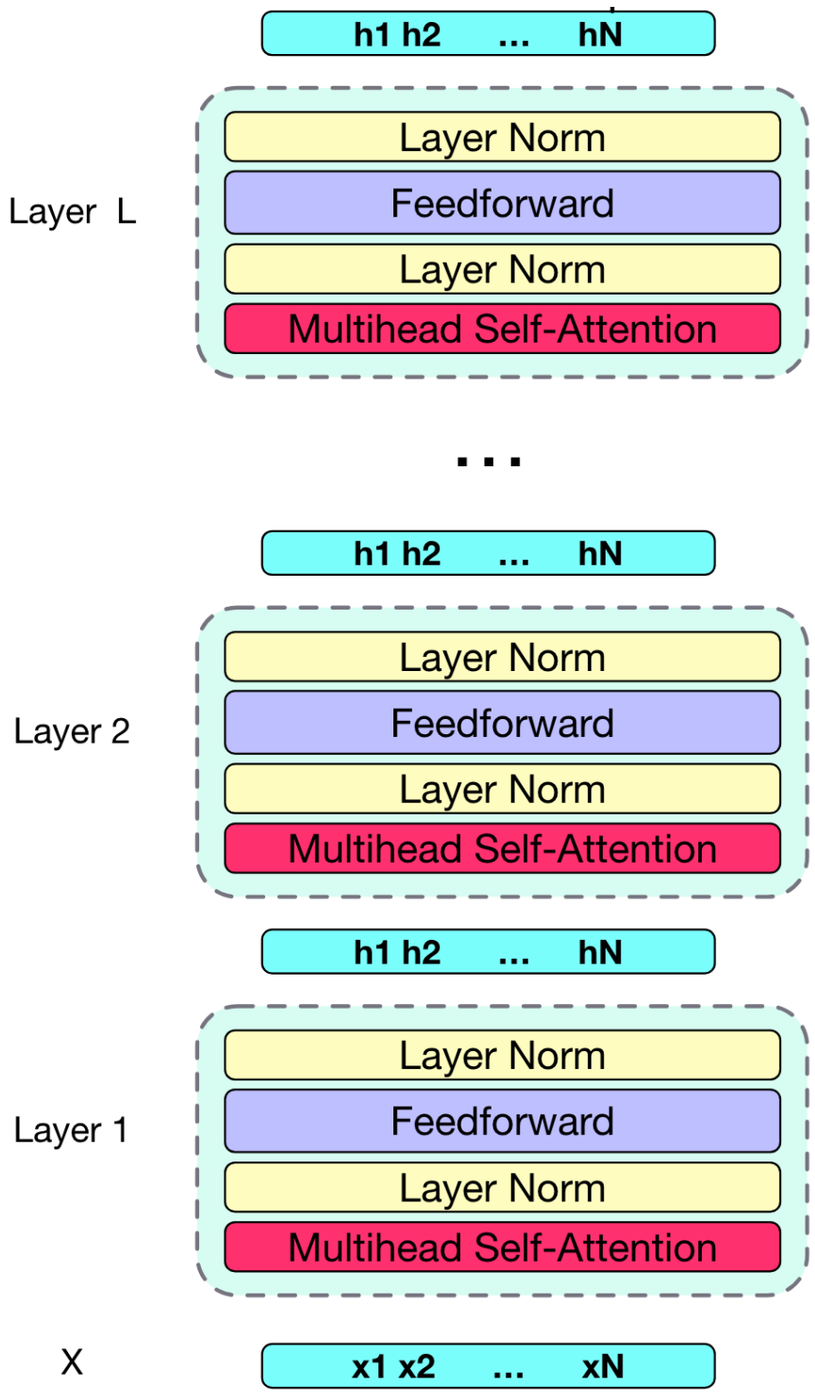
$$T^3 = \text{LayerNorm}(T^2)$$

$$T^4 = FF(T^3)$$

$$T^5 = T^4 + T^3$$

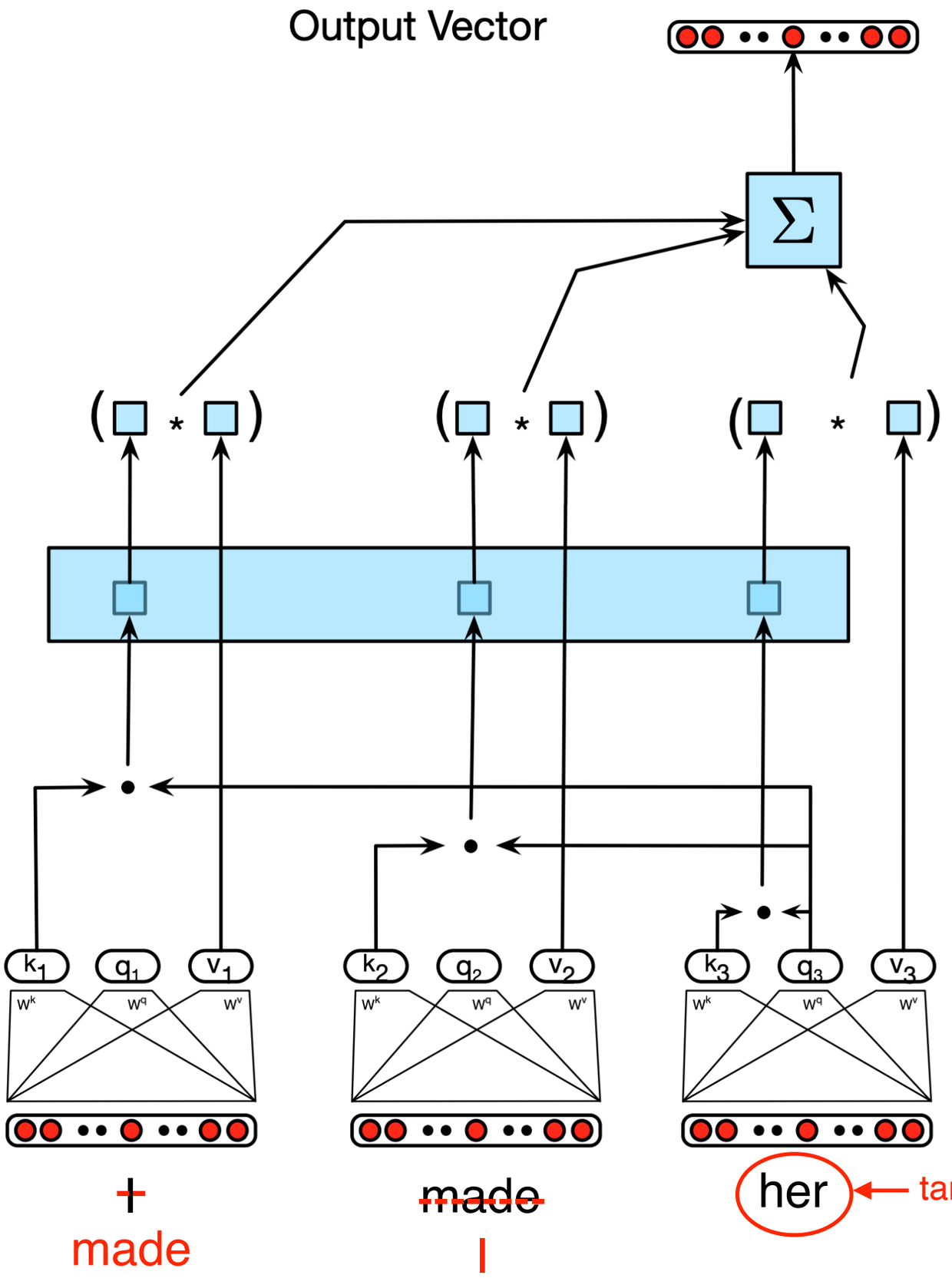
$$H = \text{LayerNorm}(T^5)$$

# Stack Them Up



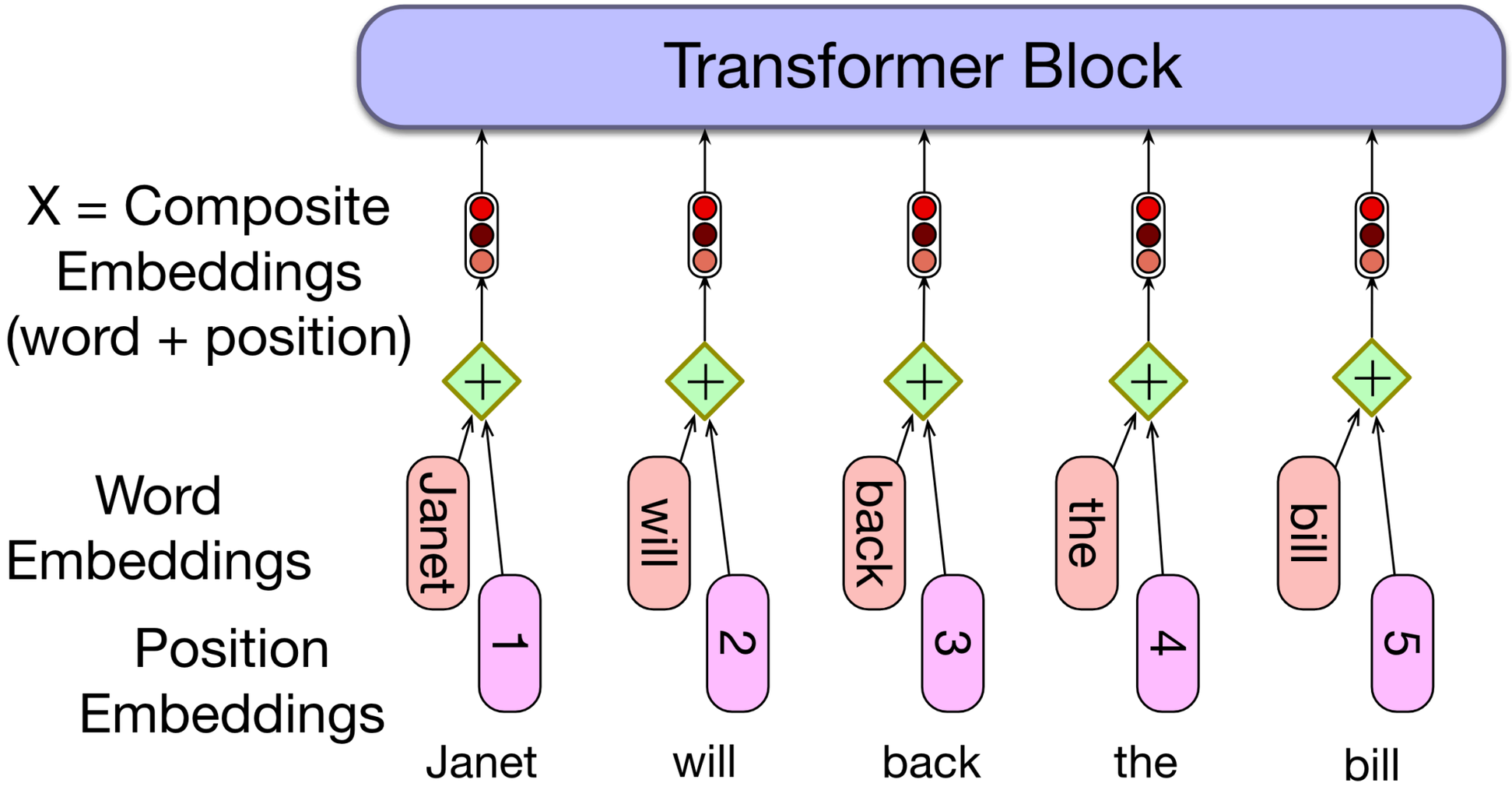

- What's better than a transformer block? More transformer blocks!
- Learn deeper representations of words
- GPT-3, a large language model, has 96 transformer blocks!

# Positional Embeddings




- What if we swap the two words *I* and *made*?
- Same output vector!
- query-key comparison $(q \cdot k_i)$ does not take word position into account
- We need positional embeddings

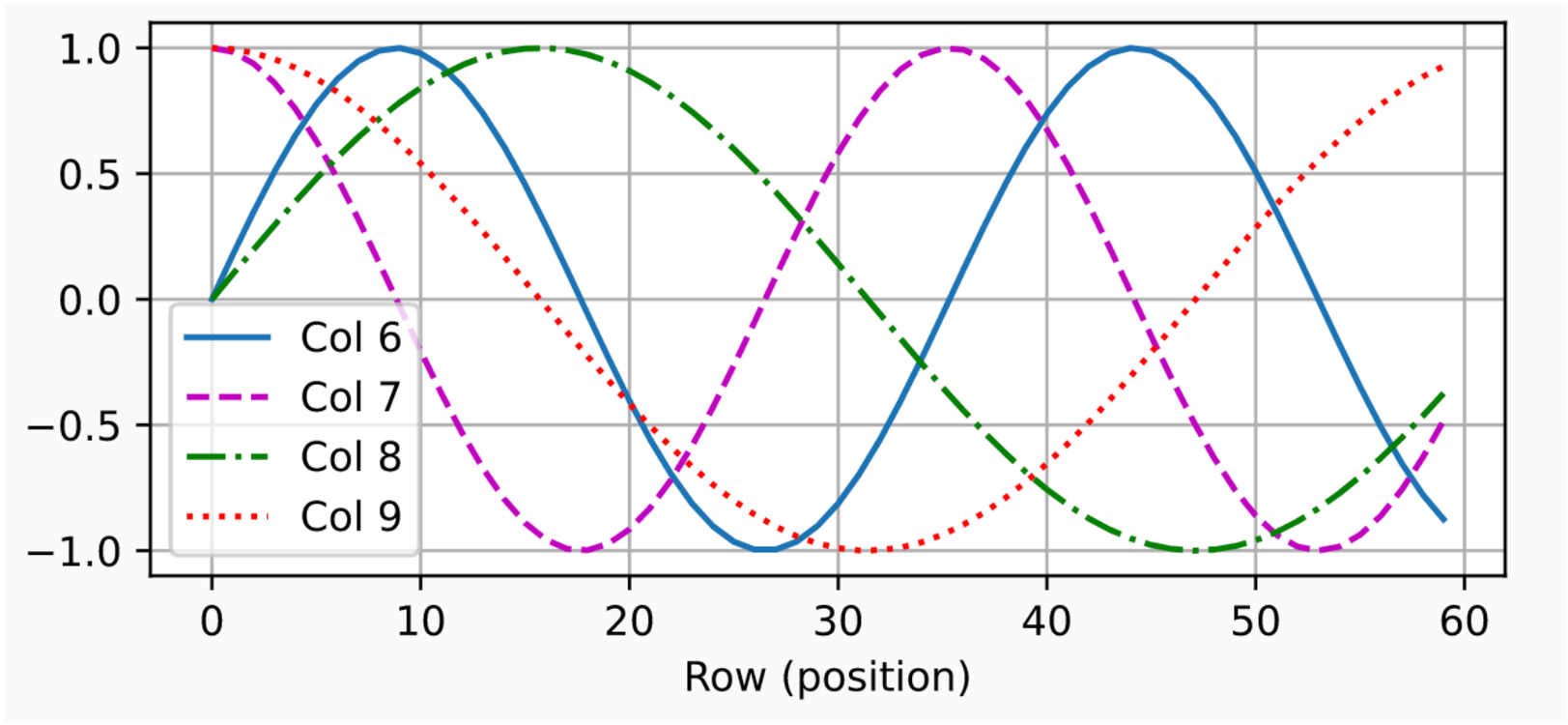
# Positional Embeddings

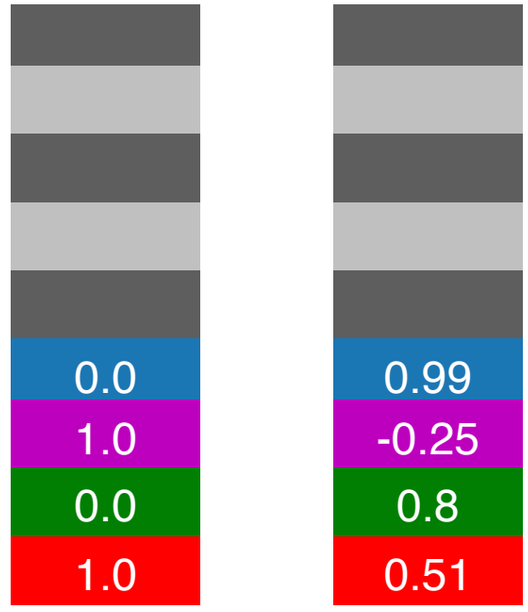



- Input embedding ($X$) is a sum of word embeddings and position embeddings

# How to Compute Them?

- Use a static function that maps each position into an embedding
  - sine and cosine functions



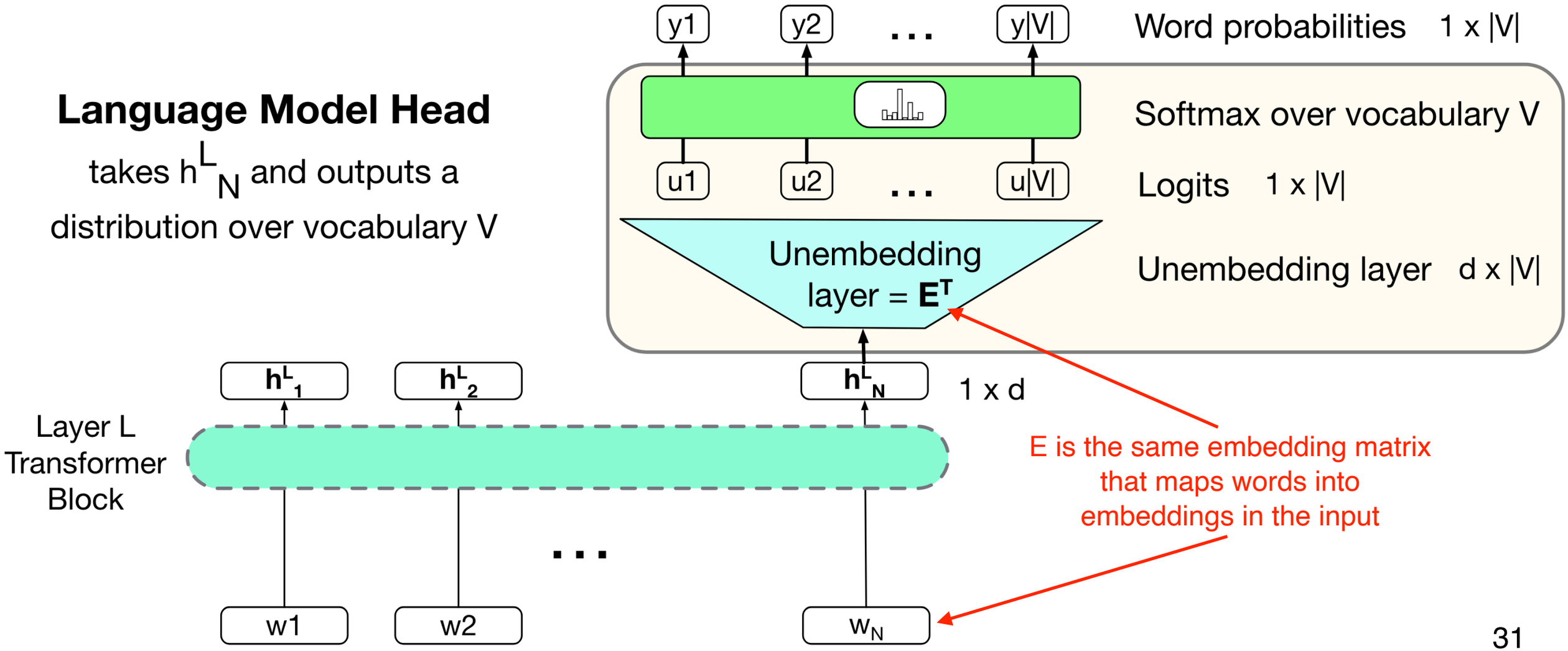

pos embedding @pos=0

pos embedding @pos=10

# Training

# Language Model Head

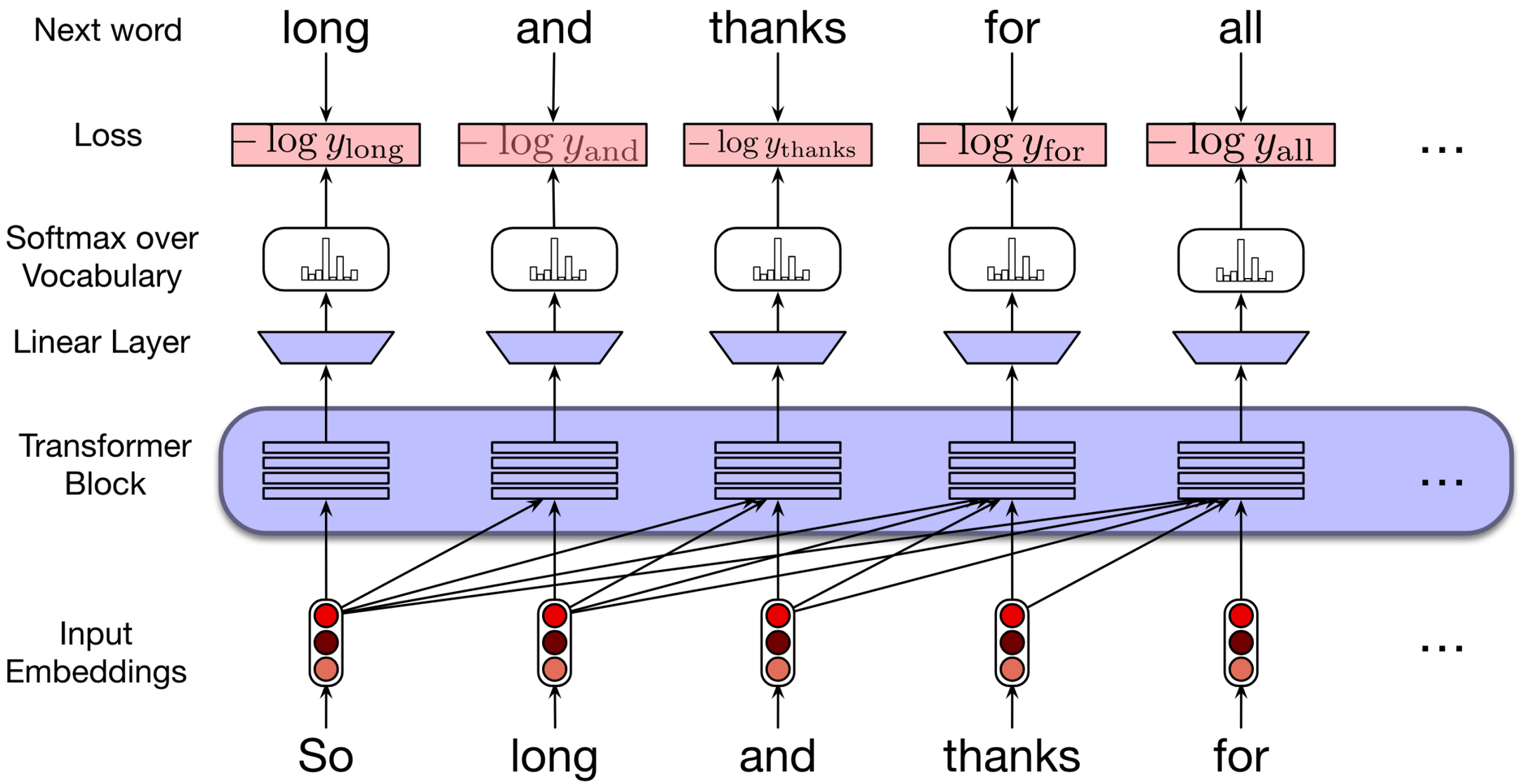
- Training objective: predict next word - how?
- Project the final output embedding ($h_i^L$) back to the vocabulary space




**Language Model Head**

takes $h^L_N$ and outputs a distribution over vocabulary V

# Training




$$= \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

# Fixed Context Language Model?

- When doing attention, for each target word we only look at a fixed number of left context words
- So it's just like an n-gram language model?
  - Takes a fixed number of previous words as context to predict the next word
- True, except the context window is much much larger
  - training = 100K word tokens; inference = 1M!

# Final Words

- Since the introduction of transformer, it has become the dominant architecture for building language models
- It scales very well:
  - Due to ease of parallelisation
  - Performance seems to keep improving when we stack more layers or introduce more heads
- Led to the development of large language models and generative AI

# Readings

- JM3 Chapter 8.1-8.5